

Lecture – 2 Complexity and Analysis of Algorithms - Part 2

M.G.Noel A.S Fernando (PhD)

Professor

NSBM/Dept. of Information Systems Engineering, UCSC



Definition

- When we compute the time complexity $T(n)$ of an algorithm, we **rarely** get an exact result, just an estimate.
- In computer science we are typically only interested in **how fast $T(n)$ is growing as a function of the input size n .**
- For example, if an algorithm increments each number in a list of length n , we might say: “This algorithm runs in $O(n)$ time and performs $O(1)$ work for each element”.
- $X=X+1$
- $X=X+1$ repeat N times
-

Common time complexities

BETTER



WORSE

- $O(1)$ constant time
- $O(\log n)$ log time
- $O(n)$ linear time
- $O(n \log n)$ log linear time
- $O(n^2)$ quadratic time
- $O(n^3)$ cubic time
- $O(2^n)$ exponential time

Big-Oh Rules

If $f(n)$ is a polynomial of degree d , then $f(n)$ is $O(n^d)$, i.e.,

1. Drop lower-order terms
2. Drop constant factors

Use the smallest possible class of functions

- “ $2n$ is $O(n)$ ” instead of “ $2n$ is $O(n^2)$ ”

Use the simplest expression of the class

- “ $3n + 5$ is $O(n)$ ” instead of “ $3n + 5$ is $O(3n)$ ”

More Examples:

- $f(n) = 7n^4 + 3n^2 + 5n + 1000$ is $O(n^4)$
 - $g(n) = 7n + 3n + 1000000$ is $O(n)$
 - $h(n) = 7n(n + \log(n))$ is $O(n^2)$
-

Questions (2 and 3)

2. Using a suitable example, show the worst-case running time of an if-then-else statement.
3. Using a suitable example, show the worst-case running time of a simple loop program segment

Question 4

- Consider the following C program segment.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n = 10; // example input size
```

```
    printf("Hello\n");
```

```
    for (int i = 0; i < n; i++)
```

```
        printf("%d\n", i);
```

```
    }
```

```
    return 0;
```

```
}
```

Calculate the worst-case time complexity of the above code.

Question 5 - Calculate the worst-case running time of the following program

Multiply for Nested Loops

Python code

```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```

C code

```
#include <stdio.h>  
int main() {  
    int n = 5; // example input size  
    for (int i = 0; i < n; i++)  
        for (int j = 0; j < n; j++)  
            printf("%d %d\n", i, j);  
    }  
    }  
    return 0;  
}
```

Question 6 - Calculate the worst-case running time of the following program

Keep the Dominant Term

When there are multiple terms, keep the *highest order*:

Python Code

```
for i in range(n):
    for j in range(n):
        print(i, j)
for k in range(n):
    print(k)
```

C code

```
#include <stdio.h>

int main() {
    int n = 5; // example input size
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d %d\n", i, j);
        }
    }
    // Separate loop: O(n)
    for (int k = 0; k < n; k++) {
        printf("%d\n", k);
    }
    return 0;
}
```


Question Number 7: Find the running time of the recursive factorial

$\text{fact}(n) = n * \text{fact}(n-1) = n * (n-1) * \text{fact}(n-2) = \dots = n * (n-1) * (n-2) * \dots * \text{fact}(1) = n!$

```
public int fact(int n) {  
    if n <= 1 return 1;  
    else return n * fact(n-1);  
}
```

Question 8 – Calculate the Running time of the following program segment.

```
int k=0;
for (int i=0; i<n; i++) {                               /* outer_loop */
    if even(i) { for (int j=0; j<n; j++)                 /* inner_loop */
        k++; };
    else k *= 2;
};
```

Question 9

- Consider an algorithm whose running time is given by $T(n) = n - 1$. Using Big-O notation, determine the time complexity of the algorithm and explain why it is classified as linear

Question 10 :calculate the time complexity

- $T(n) = n^2/2 - n/2.$

Question 11

- Prove that the function $T(n) = 3n^3 + 20n^2 + 5$ belongs to $O(n^3)$.
- Find appropriate constants $c > 0$ and $n_0 \geq 1$ such that:
- $3n^3 + 20n^2 + 5 \leq c \cdot n^3$ for all $n \geq n_0$.
- Show your calculation and justification.

Question 12

- Consider the functions $f(n) = n^3 + 20n^2 + 100n$ and $g(n) = n^3$.
- a) Using Big-O notation, determine whether $f(n) = O(g(n))$.
- (b) If possible, provide constants $c > 0$ and $n_0 \geq 1$ such that the inequality
$$f(n) \leq c \cdot g(n), \text{ for all } n \geq n_0$$
holds.
- (c) Verify your choice of constants by substituting them into the inequality.
- **hint:** You can choose $c = 121$ and $n_0 = 1$.

Question 13 :More Big-Oh questions

- Prove that $100n + 5 = O(n)$

Question 14

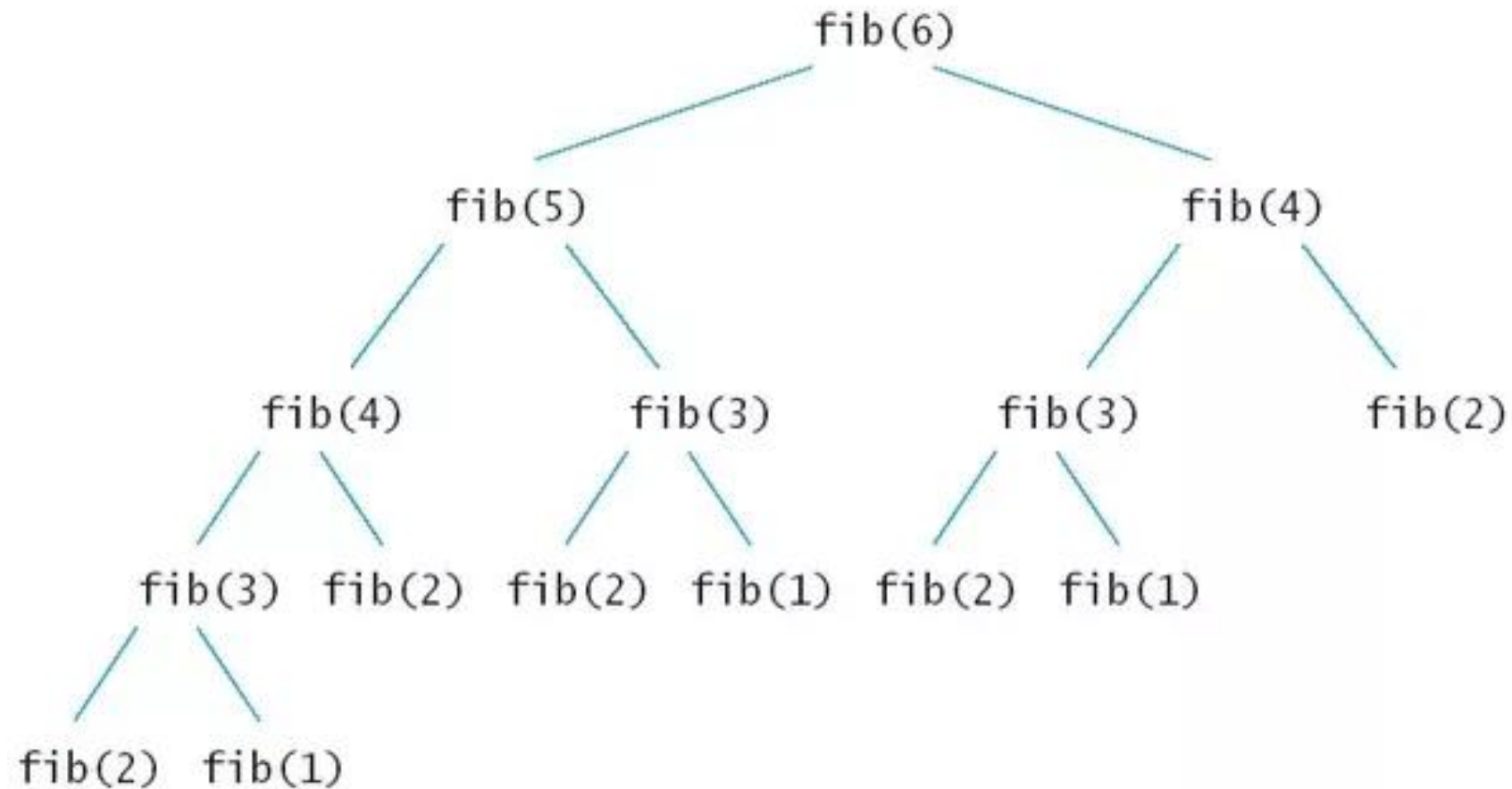
$$3n^2 - 100n + 6 = O(n^2) \quad \text{since for } c=3, \quad 3n^2 > 3n^2 - 100n + 6$$

Different big O values -Examples

- **Exponential Time — $O(2^n)$**
- An algorithm is said to have an exponential time complexity when the growth doubles with each addition to the input data set.
- A example of an exponential time algorithm is the recursive calculation of [Fibonacci](#) numbers:

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    return fibonacci(n-1) + fibonacci(n-2)
```

Fibonacci numbers:



Fibonacci numbers:

$$T(n \leq 1) = O(1)$$

$$T(n) = T(n-1) + T(n-2) + O(1)$$

Base: $n = 1$ is obvious $O(1)$ – Constant

Assume $T(n-1) = O(2^{n-1})$, therefore

$$T(n) = T(n-1) + T(n-2) + O(1) \text{ which is equal to}$$

$$T(n) = O(2^{n-1}) + O(2^{n-2}) + O(1) = O(2^n)$$

Fibonacci numbers:

- we can approximate $T(n-2) \sim T(n-1)$ as $T(n-2) \leq T(n-1)$
- $T(n) = T(n-1) + T(n-2) + c$
 $= 2T(n-1) + c$ //from the approximation $T(n-1) \sim T(n-2)$
 $= 2*(2T(n-2) + c) + c$ // $T(n)=2T(n-1)+c$, similarly $T(n-1)=2T(n-2)+1$
 $= 4T(n-2) + 3c$
 $= 8T(n-3) + 7c$
 $= 2^k * T(n - k) + (2^k - 1)*c$ /general formular
- Let's find the value of k for which: $n - k = 0$
 $k = n$
 $T(n) = 2^n * T(0) + (2^n - 1)*c$
 $= 2^n * c + 2^n * c$
i.e. $T(n) \sim 2^n$

Time Complexity Analysis of Searching Algorithms

- What are Searching Algorithms?
 - **Searching algorithms** are a category of algorithms in computer science that are designed to **find a specific item or group of items within a data structure** (like an array, list, tree, or database).
- The following questions can be answered using searching algorithms:
 - Does this element exist in the data?
 - Where is this element located?
 - How many times does it occur

Time Complexity Analysis of Searching Algorithms (cont..)

- Examples
- **Linear Search**
 - Checks each element one by one until it finds the target or reaches the end.
 - Works on unsorted data.
 - Time Complexity: $O(n)$.
- **Binary Search**
 - Works only on sorted data.
 - Repeatedly divides the search space in half to find the target.
 - Time Complexity: $O(\log n)$.
- **Tree Search**
 - Used in tree data structures like Binary Search Trees (BSTs).
 - Efficiency depends on the tree's balance.

Linear Search Algorithm

- It is used for an unsorted array.
- It mainly does one by one comparison of the item to be search with array elements.
- It takes linear or $O(n)$ Time.

Linear Search Algorithm

- Given an array, **arr** of n integers, and an integer element **x**, find whether element **x** is present in the array.
- Return the index of the first occurrence of **x** in the array, or -1 if it doesn't exist.
- **Input:** `arr[] = [1, 2, 3, 4]`, `x = 3`
Output: 2
Explanation: There is one test case with array as [1, 2, 3 4] and element to be searched as 3. Since 3 is present at index 2, the output is 2.
- **Input:** `arr[] = [10, 8, 30]`, `x = 6`
Output: -1
Explanation: The element to be searched is 6 and its not present, so we return -1.

Linear Search Algorithm

1. Start from the first element.
2. Compare the current element with the target.
3. If it matches, return the index (position).
4. If not, move to the next element.
5. Repeat steps 2–4 until:
 - You find the element (success), or
 - You reach the end (failure).

List: [4, 2, 7, 1, 3]

Target: 1

Steps:

- Compare 4 → Not 1
- Compare 2 → Not 1
- Compare 7 → Not 1
- Compare 1 → Found at index 3

Linear Search Algorithm

- Simple pseudocode

```
LinearSearch(arr, target):  
    for i from 0 to length(arr) -  
    1:  
        if arr[i] == target:  
            return i // found at  
index i  
    return -1 // not found
```

C code

```
#include <stdio.h>  
  
int linearSearch(int arr[], int n, int target) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == target) {  
            return i; // Found  
        }  
    }  
    return -1; // Not found  
}  
  
int main() {  
    int arr[] = {4, 2, 7, 1, 3};  
    int n = 5;  
    int target = 1;  
    int result = linearSearch(arr, n, target);  
    if (result != -1)  
        printf("Element found at index %d\n", result);  
    else  
        printf("Element not found\n");  
    return 0;  
}
```

Time Complexity of Linear Search

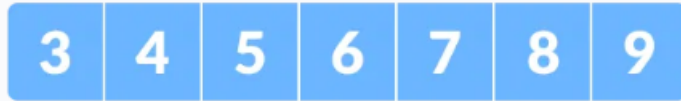
Case	Complexity	Explanation
Best Case	$O(1)$	The target element is at the first position — only one comparison is needed.
Worst Case	$O(n)$	The target element is at the last position or not present at all — every element must be checked.
Average Case	$O(n)$	On average, about $n/2$ elements will be checked, which is $O(n)$ in Big-O notation.

Binary search Algorithm

- Binary Search is a searching algorithm for finding an element's position in a sorted array.
- Binary Search Algorithm can be implemented in two ways:
- Iterative Method
- Recursive Method
- The recursive method follows [the divide and conquer](#) approach.

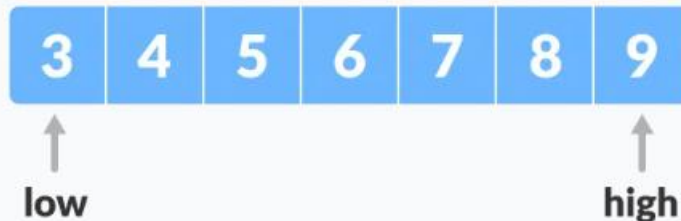
Binary search Algorithm

- The general steps for both methods are:
- **Step 1:** Consider the initial Array



3 4 5 6 7 8 9

- Let $x = 4$ be the element to be searched.
- **Step 2:** Set two pointers low and high at the lowest and the highest positions respectively



3 4 5 6 7 8 9

↑ low ↑ high

Binary search Algorithm

- **Step 3:** Find the middle position mid of the array.
- i.e. $\text{mid} = (\text{low} + \text{high})/2$ and $\text{arr}[\text{mid}] = 6$

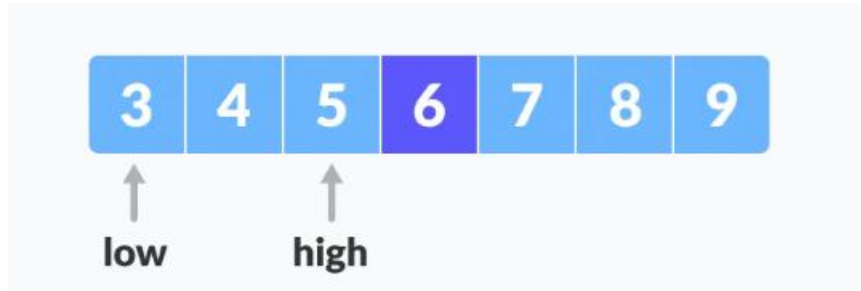


Step 4: If $x == \text{arr}[\text{mid}]$, then return mid. Else, compare the element to be searched with $\text{arr}[\text{mid}]$.

Step 5: If $x > \text{arr}[\text{mid}]$, compare x with the middle element of the elements on the right side of $\text{arr}[\text{mid}]$. This is done by setting low to $\text{low} = \text{mid} + 1$.

Step 6: Else, compare x with the middle element of the elements on the left side of $\text{arr}[\text{mid}]$. This is done by setting high to $\text{high} = \text{mid} - 1$

Binary search Algorithm



Step 7: Repeat steps 3 to 6 until low meets high



$x = 4$ is found

Binary search Algorithm

- Iterative Method

```
do until the pointers low and high meet each other.  
    mid = (low + high)/2  
    if (x == arr[mid])  
        return mid  
    else if (x > arr[mid]) // x is on the right side  
        low = mid + 1  
    else // x is on the left side  
        high = mid - 1
```

- Recursive

```
binarySearch(arr, x, low, high)  
    if low > high  
        return False  
    else  
        mid = (low + high) / 2  
        if x == arr[mid]  
            return mid  
        else if x > arr[mid] // x is on the right side  
            return binarySearch(arr, x, mid + 1, high)  
        else // x is on the left side  
            return binarySearch(arr, x, low, mid - 1)
```


Binary search Algorithm -Time complexity

- Each iteration cuts the problem size in half:
- After 1st iteration: $n/2$
- After 2nd iteration: $n/4$
- After 3rd iteration: $n/8$
-
-
- After k iterations: $n/2^k$
- we stop $n/2^k=1$
- In **binary search**, you keep halving the input size n .
- After 1 step $\rightarrow$ you have $n/2$ elements left.
- After 2 steps $\rightarrow$ you have $n/4$ elements left.
- After k steps $\rightarrow$ you have $n/2^k$ elements left.
- You continue this process **until there is only 1 element left** to check

$$n/2^k = 1 \implies 2^k = n \implies k = \log_2 n.$$